

BFP Rosario GIS: System at Hardware

Ipinapaliwanag ng dokumentong ito kung **paano gumagana** at **paano ginagamit** ang buong BFP Rosario GIS system. May tatlong bahagi ito:

1. Ang **app** (phone, computer, at website)
2. Ang **Firebase**, ang "backend" o online database
3. Ang **ESP32 incident alarm**, ang device sa fire station na nagpapatunog ng siren kapag may nag-report

Isinulat ito para sa mga estudyante, kaya ipinapaliwanag muna ang ideya bago ang technical na detalye.

1. Pangkalahatang ideya (System overview)

1.1 Ang problema na sinolusyunan

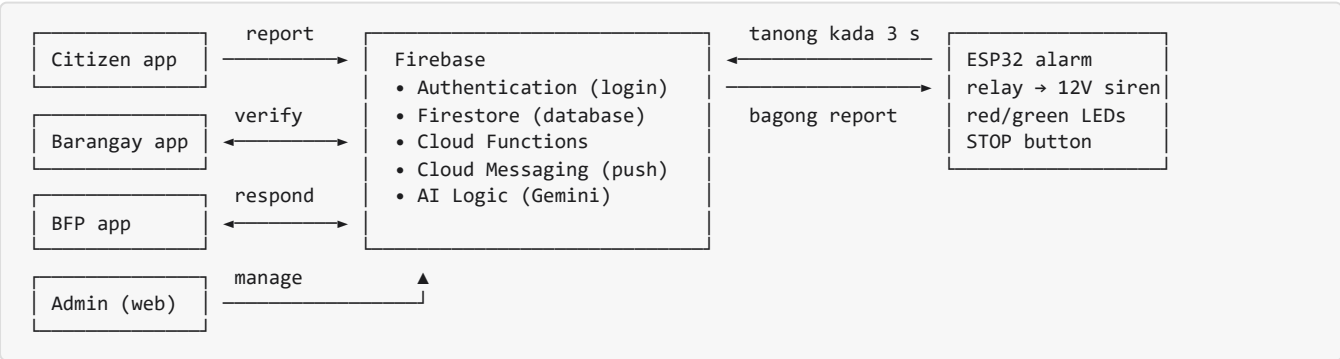
Kapag may sunog o emergency sa Rosario, Batangas, kailangang malaman agad ng BFP (Bureau of Fire Protection) kung **ano**, **gaano kalala**, at **saan eksakto**. Sa system na ito:

- Ang **citizen** ay nagre-report gamit ang app, kasama ang **GPS location** at **litrato**.
- Lumalabas agad ang report sa **mapa** ng barangay staff, BFP, at admin.
- Sabay nito, **tumutunog ang siren sa fire station**, kahit walang nakabukas na app.

1.2 Ang tatlong bahagi

Bahagi	Ginamit	Saan tumatakbo
BFP Rosario GIS app	Flutter (Android, iOS, web, Windows, macOS, Linux)	Phone ng citizens, device ng barangay at BFP, admin website
Backend	Firebase: Authentication, Firestore, Cloud Functions, Cloud Messaging, Hosting, AI Logic (Gemini)	Google Cloud, project <code>gis-cross-platform</code>
Incident alarm	ESP32 + relay + 12 V siren, naka-12 V UPS	BFP Rosario fire station

Isipin ang **Firebase** bilang **gitnang "bulletin board"**. Lahat ng bahagi ay nagbabasa at nagsusulat dito. Hindi direktang nag-uusap ang app at ang ESP32. Pareho silang dumadaan sa Firebase.



Ang mapa ay gawa sa `flutter_map` at OpenStreetMap, at ang address ay hinahanap gamit ang Nominatim. Nakasentro ang mapa sa Rosario, at tinatanggihan ang report na nasa labas ng munisipyo.

2. Ang app at ang backend

2.1 Mga user at ang kanilang role

Bawat account ay may **role**, na naka-save sa Firestore sa `users/{uid}`, field `role`. Ang role ang nagsasabi kung ano ang puwedeng gawin ng account. Ang **Security Rules** ng Firestore ang nagbabantay nito.

Role	Value ng role	Ano ang ginagawa
Citizen / User	resident	Nagre-register gamit ang government ID. Kapag verified na, puwede nang mag-report na may location at litrato. Sariling reports lang ang nakikita
Barangay Staff	barangay	Tinitingnan ang mga Pending na report at ginagawang Verified. Kapag na-verify, naaabisuhan ang lahat ng BFP account
BFP Personnel	bfp	Tumatanggap ng alerts, at ina-update ang report habang rumeresponde (Dispatched → On Scene → Contained → Resolved → Closed)
System Admin (web)	admin	Nag-a-approve o nagre-reject ng citizen ID, nagro-route ng incident sa BFP, at gumagawa ng barangay, BFP at resident accounts
Alarm device	bfp	Ang sariling account ng ESP32. Nagbabasa lang ito ng incidents

2.2 Paano ginagamit: step by step

Citizen 1. Mag-register sa app at i-scan ang government ID. 2. Hintayin na i-approve ng admin ang account. 3. Para mag-report: buksan ang report form, piliin ang type, isulat ang detalye, at piliin ang urgency (Low, High, Critical). 4. I-set ang location: gamitin ang GPS, i-tap ang mapa, o i-type ang address. 5. Magdagdag ng litrato kung meron, tapos i-submit.

Barangay Staff 1. Buksan ang verification queue. 2. Tingnan ang Pending na report (detalye, litrato, lugar sa mapa). 3. I-verify kung totoo. Makakatanggap agad ng push notification ang BFP.

BFP Personnel 1. Makakakita ng "New incident report" alert sa app, at tutunog ang siren sa station. 2. Pindutin ang **STOP** sa alarm box para patahimikin ang siren. 3. Sa app, i-update ang status habang rumeresponde: Dispatched, On Scene, Contained, Resolved, Closed. Sa Resolved at Closed, maglagay ng response notes.

Admin (sa website) 1. I-approve o i-reject ang mga bagong citizen. 2. I-route ang incident sa BFP kung kailangan. 3. Gumawa ng accounts, kasama ang account ng **alarm device** (role: BFP Personnel).

2.3 Pag-verify ng citizen

- Nagre-register ang citizen at ini-scan ang government ID (Philippine National ID, Driver License, Passport, UMID, o PhilHealth ID).
- Tinitingnan ito ng AI (Gemini) kung malinaw at mukhang totoong ID. **Advisory lang ito.**
- Ang **admin** ang nag-a-approve o nagre-reject. Ang verified citizen lang ang makakapagbukas ng report form.

Ginagawa ito para mabawasan ang **prank o pekeng report**.

2.4 Ano ang laman ng isang report

Kapag nag-submit ang citizen, gumagawa ang app ng bagong document sa `incidents` collection ng Firestore. Ito ang mga pangunahing field:

Field	Ibig sabihin
type, description	Uri ng insidente at detalye
priority	Low, High o Critical (tinatanggap din ng rules ang Medium para sa lumang data)
latitude, longitude	Lokasyon: galing GPS, tap sa mapa, o address search
barangayName, barangayId	Barangay na natukoy mula sa lokasyon
evidenceImage	Litrato (optional), naka-save sa loob ng document
aiAssessment	Resulta ng pag-assess ng Gemini sa litrato
uid, reporterId, reporterName, phone, address	Impormasyon ng nag-report
status	Laging Pending kapag bagong gawa
createdAt, updatedAt	Oras galing sa server. createdAt ang binabantayan ng ESP32

Tungkol sa priority at AI: kapag may litrato, tinatantya ng Gemini ang severity nito. Ang severity na iyon ang **pinakamababang priority** na puwedeng piliin ng citizen. Halimbawa, kapag sinabi ng AI na Critical, hindi puwedeng i-submit bilang Low. Pinapatupad din ito ng Security Rules.

2.5 Ang buhay ng isang report (status)



Naka-record kung sino ang nag-update sa bawat hakbang.

2.6 Paano naaabisuhan ang mga tao

Alert	Paano gumagana	Limitasyon
Alert dialog sa app	Sa BFP at admin screens, binabantayan ng app ang mga Pending report at tumutunog kada 2 segundo hanggang i-acknowledge	Gumagana lang habang bukas ang app
Push notification	Kapag na-verify ng barangay ang report, may notifications document na nagagawa para sa bawat BFP user. Ang Cloud Function na sendBfpEmergencyPush ang nagpapadala ng push (FCM)	Kailangang naka-install ang app at naka-on ang notifications
Siren ng ESP32	Tinatanong ng alarm box ang Firestore kada 3 segundo kung may bagong report, at pinapatunog ang siren	Kailangan ng Wi-Fi at kuryente (sagot ng UPS ang brownout)

Ang ESP32 ang **"laging gising"** na alert ng station. Gumagana ito kahit walang nakabukas na phone o computer.

2.7 Paano kumokonekta ang ESP32 sa Firebase

- May **sariling email at password account** ang ESP32, na ginawa ng Super Admin sa **Accounts** page na may role na **BFP Personnel**. Kailangan ito kasi BFP, barangay at admin lang ang pinapayagan ng rules na magbasa ng lahat ng incident.
- Gumagamit ito ng **Firebase REST API** (parang pag-access sa website gamit ang HTTPS). Walang Firebase app na naka-install sa ESP32.
- **Nagbabasa lang** ito ng incidents. Hindi ito nagsusulat sa Firestore.

- **Tumutunog agad ito sa bawat bagong report**, kahit `Pending` pa at hindi pa verified ng barangay. Ginagawa ito para makapaghandag agad ang station.
- Ang email at password ng device ay nasa `hardware/esp32_incident_alarm/secrets.h`. Hindi ito naka-upload sa git, para hindi makita ng iba.

3. Ang ESP32 incident alarm (hardware)

Ang incident alarm ay isang maliit na box sa fire station. Laging naka-on ito (may 12 V UPS para sa brownout), binabantayan ang Firebase, at pinapatunog ang 12 V siren kapag may bagong report.

3.1 Mga piyesa (components)

Piyesa	Trabaho
ESP32 Dev Module (ESP32-D0WD-V3)	Ang "utak": Wi-Fi, pagtanong sa Firebase, LEDs, button, at kontrol ng relay
SRD-05VDC-SL-C 1-channel relay module	Parang switch na kinokontrol ng ESP32 . Ito ang nagbubukas at nagsasara ng kuryente papunta sa siren
LTE-1101J siren, 12 V DC	Ang malakas na tunog ng alarm
12 V DC mini UPS	Pangunahing power. Tuloy ang alarm kahit brownout
Buck converter (LM2596-type), naka-set sa 5.0 V	Pinapababa ang 12 V ng UPS papuntang 5 V para sa ESP32
Red LED + 220 ohm resistor	Ilaw ng alarm, at ng Wi-Fi setup (kumukurap)
Green LED + 220 ohm resistor	Ilaw ng "ready" o online
4-pin push button	STOP button: pinapatahimik ang alarm, at binubuksan o sinasara ang Wi-Fi setup kapag hinawakan nang matagal
Breadboard at jumper wires	Mga koneksyon

Optional ang HW-131 / MB102 breadboard power module. Puwede itong pamalit sa buck converter kapag nagtest, pero umiinit ito sa 12 V, kaya buck converter ang ginamit sa final setup.

Bakit may relay? Ang GPIO pin ng ESP32 ay 3.3 V lang at kaunting kuryente ang kaya. Ang siren ay 12 V at mas malakas humigop ng kuryente. Kaya ang ESP32 ay "pumipindot" lang sa relay, at ang relay ang nagkakabit ng 12 V sa siren.

3.2 Pin assignment

ESP32 pin	Nakakabit sa	Paalala
GPIO25	Relay IN	Direktang wire. LOW = relay ON (siren), HIGH = relay OFF
GPIO14	Red LED (+) sa 220 ohm	Ang LED (-) ay sa GND
GPIO27	Green LED (+) sa 220 ohm	Ang LED (-) ay sa GND
GPIO33	STOP button, isang paa	Ang dayagonal na paa ay sa GND. Internal pull-up ang gamit kaya walang resistor
5V	Buck converter +OUT (5.0 V)	Power ng ESP32 kapag galing UPS
3V3	Relay VCC	Tingnan ang 3.4 kung bakit sa 3V3 ang relay
GND	Buck -OUT, relay GND, LED (-), button	lisa ang GND ng lahat

Naka-reserve ang GPIO26 sa firmware para sa optional na buzzer. Walang buzzer sa final build, kaya siren lang ang tunog.

3.3 Wiring

```
POWER
UPS 12V (+)  ──┬──> Buck +IN
                └──> Relay COM (公共, gitnang turnilyo)
UPS 12V (-)  ──┬──> Buck -IN
                └──> Siren BLACK

Buck +OUT (5.0 V) ──> ESP32 5V
Buck -OUT (GND)  ──> ESP32 GND

RELAY
Relay VCC ──> ESP32 3V3
Relay GND ──> GND
Relay IN  ──> GPIO25
Relay NO (常开) ──> Siren RED
Relay NC (常闭) ──> walang nakakabit

LEDS AT BUTTON
GPIO14 ──> 220 ohm ──> Red LED ──> GND
GPIO27 ──> 220 ohm ──> Green LED ──> GND
GPIO33 ──> STOP button (dayagonal na paa) ──> GND
```

Chinese ang nakasulat sa screw terminals ng relay: 常开 = **NO** (normally open), 公共 = **COM** (common), 常闭 = **NC** (normally closed). Nasa **NO** ang siren, kaya tahimik ito hangga't hindi naka-ON ang relay.

3.4 Mga natutunan habang nagte-test

- **I-set muna sa 5.0 V ang buck converter bago ikabit ang ESP32.** Madalas mas mataas sa 5 V ang default nito at puwedeng masunog ang ESP32. May button ang display para lumipat sa IN o OUT reading.
- **Relay ang nagpapagana sa siren, hindi ang GPIO.** Masyadong malakas humigop ng kuryente ang siren at 12 V ito.
- **Sa 3V3 ang VCC ng relay, hindi 5 V.** Walang H/L trigger jumper ang relay na ito. Noong nasa 5 V ang VCC, kaya ng 3.3 V signal ng ESP32 na i-ON ang relay pero hindi ito ma-OFF nang tuluyan, kaya hindi tumitigil ang siren. Sa 3V3, maayos na nag-o-ON at OFF ang relay (LOW = ON, HIGH = OFF).
- **Pinahina ang Wi-Fi transmit power.** Iisang 3.3 V supply ang gamit ng relay coil at ng Wi-Fi ng ESP32. Sa buong lakas ng Wi-Fi, umiilaw ang LED ng relay pero hindi gumagalaw ang coil. Ibinaba ng firmware ang

Wi-Fi power sa 8.5 dBm, at gumagana na ang siren kahit online. Kung malayo ang router at nawawala ang koneksyon, kakailanganin ng transistor o ng 3.3 V-trigger relay/MOSFET module.

- **Dayagonal na paa ng 4-pin button ang gamitin.** Laging magkakonekta ang dalawang paa sa iisang gilid, kaya iisipin ng ESP32 na laging pinipindot ang button, at bubukas ang Wi-Fi setup sa bawat boot.
- **Iisang GND.** Magkakonekta ang GND ng ESP32, relay at buck converter.
- **Huwag pagsabayin ang USB at ang buck sa ESP32.** Kapag mag-a-upload ng code gamit ang USB, tanggalin muna ang wire ng buck +OUT → ESP32 5V.

3.5 Mga ilaw at kontrol

Nakikita mo	Ibig sabihin
Steady green	Ready: online at nagbabantay ng bagong report
Mabagal na kurap ng green	Kumokonekta sa Wi-Fi/Firebase, o may error
Kumukurap ang red, patay ang green	Bukas ang Wi-Fi setup hotspot
Red + siren	May bagong incident report. Pindutin ang STOP para tumahimik

Gawin	Mangyayari
Pindutin ang STOP	Titigil ang alarm hanggang sa susunod na bagong report
Hawakan ang STOP nang 5 segundo	Bubukas ang Wi-Fi setup hotspot na <code>BFP-Alarm-Setup</code> (kumukurap ang red)
Hawakan ulit nang 5 segundo	Sasara ang hotspot, papatayin ang radio nito, at babalik sa green
Hawakan ang STOP habang nag-o-on	Bubukas agad ang Wi-Fi setup

Kusa ring nagsasara ang setup hotspot pagkalipas ng 3 minuto kung walang na-save.

3.6 Paano palitan ang Wi-Fi

1. Hawakan ang STOP nang 5 segundo. Kukurap ang red LED.
2. Sa phone, kumonekta sa `BFP-Alarm-Setup` (ang password ay `SETUP_AP_PASSWORD` sa `secrets.h`).
3. Kusang bubukas ang setup page. Kung hindi, buksan ang `http://192.168.4.1` sa browser.
4. Pindutin ang **Configure WiFi**, piliin ang network (2.4 GHz lang), ilagay ang password, at i-Save.
5. Kokonekta ang ESP32 sa bagong network, mamamatay ang hotspot, at magiging steady green.

4. Ang firmware (code ng ESP32)

File	Para saan
<code>hardware/esp32_incident_alarm/esp32_incident_alarm.ino</code>	Ang totoong alarm firmware
<code>hardware/esp32_incident_alarm/secrets.h</code>	Password ng setup hotspot, Firebase API key at project ID, email at password ng device account. Hindi naka-upload sa git; kopyahin mula sa <code>secrets.example.h</code>
<code>hardware/esp32_wiring_test/esp32_wiring_test.ino</code>	Pang-test ng wiring nang walang Wi-Fi: green pagka-on, pindot = red + siren, pindot ulit = green

Libraries: **ArduinoJson 7** at **WiFiManager (tzapu)**. Board: **ESP32 Dev Module**.

4.1 Paano gumagana ang firmware

May **dalawang core** (dalawang "processor") ang ESP32. Hinati ang trabaho sa dalawa para hindi ma-freeze ang siren o ang STOP button kapag mabagal ang internet.

Core 0: network task (kada 3 segundo)

1. Kumonekta sa naka-save na Wi-Fi gamit ang WiFiManager, o buksan ang setup hotspot.
2. Mag-login sa Firebase Authentication gamit ang device account (`accounts:signInWithPassword`). Nire-renew ang ID token kada 50 minuto.
3. Sa unang matagumpay na query, tandaan ang `createdAt` ng pinakabagong report bilang simula. Kaya hindi tumutunog ang mga lumang report tuwing nag-o-on ang device.
4. Magtanong sa Firestore sa `incidents` : "may report ba na `createdAt > lastSeen ?`", naka-order ayon sa `createdAt`, at `priority`, `type`, `barangayName` at `createdAt` lang ang kinukuha.
5. Sa bawat bagong report, i-update ang `lastSeen` at mag-request ng alarm ayon sa severity.

Core 1: alarm loop (kada 10 milliseconds)

- Simulan ang alarm (relay ON, red LED) kapag may bagong report. Kapag may mas seryosong report, nag-a-upgrade ang alarm; hindi ito bumababa.
- Basahin ang STOP button (50 ms debounce; 5 segundong hawak para sa Wi-Fi setup).
- Paandarin ang LEDs ayon sa table sa 3.5.
- Tanggapin ang Serial test commands.

Bakit "polling" kada 3 segundo? Mas simple ito para sa ESP32 kaysa sa real-time listener, at maliit ang data na kinukuha kada tanong. Ang pinakamatagal na delay mula report hanggang siren ay mga 3 segundo, dagdag pa ang oras ng internet.

4.2 Severity

Ang `priority` ng report ang nagdedesisyon ng pattern ng alarm:

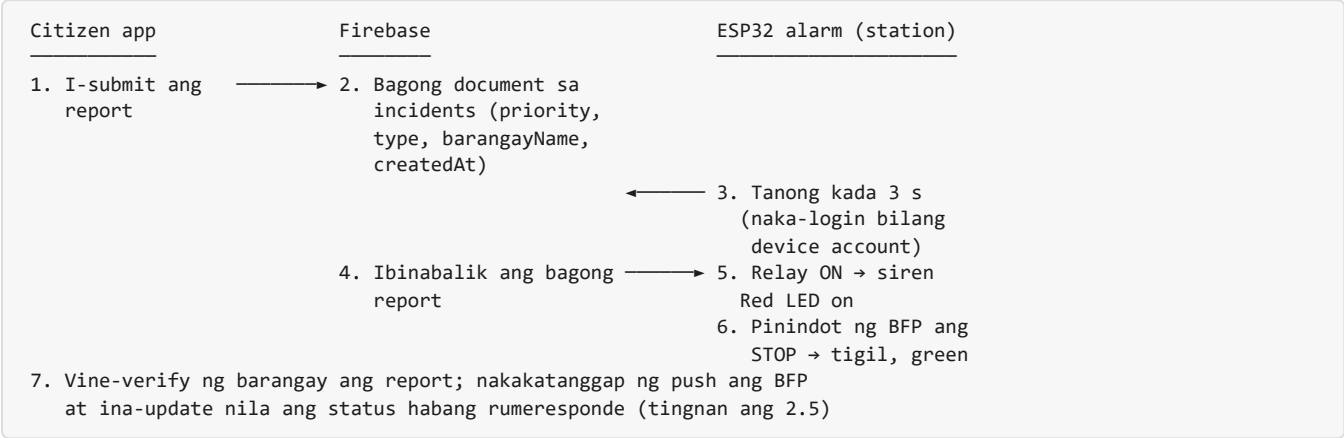
priority	Severity	Red LED
Critical	Critical	Steady
High (o hindi kilala)	High	Mabilis na kurap
Medium / Low	Low	Mabagal na kurap

Pareho ang tunog ng siren sa lahat ng severity, kasi iisang tono lang ang LTE-1101J. Magagamit din ang mga pattern na ito sa buzzer sa GPIO26 kung magdadagdag nito.

4.3 Pag-upload at pag-test

- **Arduino IDE:** buksan ang sketch, piliin ang board na **ESP32 Dev Module** at port na **COM7**, tapos i-Upload.
- **Terminal:** `arduino-cli compile --upload -p COM7 --fqbn esp32:esp32:esp32 hardware/esp32_incident_alarm` (kasama na ang `arduino-cli` sa Arduino IDE).
- **Serial Monitor** (115200 baud, line ending **New Line**): i-type ang `1`, `2` o `3` at Enter para sa Low, High o Critical na test alarm, at `0` para patigilin. Isang digit lang bawat linya, para hindi makapagpatunog ng siren ang electrical noise sa serial pin.
- Ipinapakita ng boot log ang dahilan ng pag-restart (halimbawa `BROWNOUT` kapag bumaba ang power).

5. Buong daloy: mula report hanggang siren



Ang pinakamatagal na delay mula sa pag-submit hanggang tumunog ang siren ay mga **3 segundo**, dagdag pa ang oras ng internet.

6. Troubleshooting

Problema	Posibleng dahilan	Ayos
Laging mabagal na kurap ang green	Walang Wi-Fi o hindi maka-login sa Firestore	Tingnan ang Serial log. Hawakan ang STOP nang 5 segundo para pumili ng ibang network. 2.4 GHz lang
Kumukurap ang red sa bawat boot (kusang bumubukas ang setup)	Laging "pinipindot" ang STOP button	Gamitin ang dayagonal na paa ng button
Hindi tumitigil ang siren	Nasa 5 V ang relay VCC, o nasa NC ang siren	Relay VCC sa 3V3; siren sa NO
Umiilaw ang relay pero walang siren	Kulang ang 3.3 V para sa coil, o maluwa ang COM/NO wire	Higpitan ang turnilyo; panatiliing mahina ang Wi-Fi power, o gumamit ng transistor / 3.3 V-trigger module
Walang ilaw ang display ng buck	Naka-OFF ang UPS, baligtad ang polarity, o maluwa ang wire	I-ON ang UPS, i-check ang + at -, at tanso ang ipitin (hindi goma)
permission denied sa log	Walang BFP Personnel role ang device account	I-set ang role sa admin panel
Walang COM7 kapag mag-a-upload	Tanggal ang USB o pang-charge lang ang cable	Gumamit ng data cable; isara ang Arduino Serial Monitor

7. Technologies used (Mga ginamit na technology)

7.1 App framework

Technology	Para saan
Flutter (Dart)	Isang code lang para sa Android, iOS, web, Windows, macOS at Linux. Ito ang dahilan kung bakit "cross-platform" ang app
<code>provider</code>	State management: pag-manage ng data sa pagitan ng mga screen
<code>camera</code> , <code>image_picker</code> , <code>image</code>	Pagkuha at pag-compress ng litrato ng ID at ng incident
<code>http</code>	Pag-connect sa mga online API, gaya ng Nominatim
<code>intl</code>	Format ng petsa at oras
<code>url_launcher</code>	Pagbukas ng tawag o link, halimbawa ang hotline ng fire station

7.2 GIS at mapa

Technology	Para saan
flutter_map	Ang mismong mapa sa app at website. Libre at open source (hindi Google Maps)
OpenStreetMap tiles	Ang itsura ng mapa: mga kalsada at lugar. Libreng map data galing sa OpenStreetMap
<code>latlong2</code>	Coordinates (latitude at longitude) ng mga pin sa mapa

May kulay ang mga marker ng incident depende sa priority at status: pula ang Critical, amber ang High, asul ang Verified, at berde ang Resolved/Closed. Makikita rin sa mapa ang fire station at ang coverage circle nito.

7.3 GPS at location

Technology	Para saan
geolocator	Kinukuha ang GPS location ng phone o browser ng citizen kapag nagre-report
Nominatim (OpenStreetMap)	Geocoding: ginagawang address ang coordinates (reverse), at ginagawang coordinates ang na-type na address (search). Dito rin nalalaman kung anong barangay ang report
<code>geocoding</code>	Backup na address lookup sa phone

Puwede ring i-tap ng citizen ang mapa para ilagay ang pin. Tinatanggihan ng security rules ang report na nasa labas ng Rosario.

7.4 AI (Artificial Intelligence)

Technology	Para saan
Google Gemini (<code>gemini-3.5-flash-lite</code>) gamit ang Firestore AI Logic (<code>firebase_ai</code>)	1. Pag-check ng government ID ng citizen sa registration: kung malinaw at mukhang totooong government ID. 2. Pag-assess ng litrato ng incident: tinatantya ang severity (Low / High / Critical) at ang type ng insidente. Ang severity galing sa AI ang pinakamababang priority na puwedeng piliin ng citizen
Gemini API (<code>gemini-2.0-flash</code>) sa Cloud Function	Backup na ID review sa server side (<code>reviewCitizenIdWithAi</code>)
Decision tree (<code>lib/core/models/incident_risk.dart</code>)	Risk score ng incident na nakikita ng staff. Hindi ito AI model. Rule-based ito na isinulat sa code, at hindi nito binabago ang <code>priority</code>

Advisory lang ang resulta ng AI. Admin pa rin ang nag-a-approve o nagre-reject ng bawat citizen.

7.5 Backend (Firebase)

Technology	Para saan
Firebase Authentication	Accounts at login: email/password, Google Sign-In, at Facebook Login
Cloud Firestore	Database ng <code>users</code> , <code>incidents</code> , <code>notifications</code> at <code>activity_logs</code> . Dito rin binabasa ng ESP32 ang mga bagong report
Firestore Security Rules	Access depende sa role, at pag-check ng report (fields, priority, sakop ng Rosario, laki ng litrato)
Cloud Functions	Server code: pagpapadala ng emergency push notification sa BFP (<code>sendBfpEmergencyPush</code>) at ID review
Firebase Cloud Messaging (FCM)	Push notifications sa phone ng BFP personnel
Firebase Hosting	Dito naka-host ang website (web version ng app)

Hindi ginagamit ang Firebase Storage kasi kailangan nito ng bayad na Blaze plan. Kaya naka-save ang mga litrato mismo sa loob ng Firestore document bilang compressed image.

7.6 Hardware at firmware

Technology	Para saan
Arduino (C++) sa ESP32	Ang firmware ng alarm
WiFiManager	Pagpili ng Wi-Fi gamit ang phone, sa hotspot na <code>BFP-Alarm-Setup</code>
ArduinoJson	Paggawa at pagbasa ng JSON na pinapadala at natatanggap galing Firebase
Firebase REST APIs	Pag-login ng device (Identity Toolkit) at pagbasa ng <code>incidents</code> (Firestore <code>runQuery</code>)
FreeRTOS tasks (kasama na sa ESP32)	Pagpapatakbo ng network task at alarm loop sa magkahiwalay na core